

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з навчального проекту №2
з дисципліни «Методи та технології паралельного програмування»

Тема: «Створення розподілено-паралельної чат-системи»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Лесик Валентин Олександрович

Дата здачі: 14.05.2026

Мета роботи. Освоїти принципи побудови розподілених клієнт-серверних систем обміну повідомленнями. Навчитись застосовувати засоби паралельного програмування для одночасного обслуговування багатьох клієнтів. Дослідити проблеми синхронізації, конкурентного доступу до спільних ресурсів та мережевої взаємодії. Сформувані практичні навички проєктування надійних сервісів обміну повідомленнями у реальному часі. Проаналізувати продуктивність, масштабованість і відмовостійкість запропонованого рішення.

Завдання. Необхідно розробити чат-систему з архітектурою клієнт-сервер, у якій сервер одночасно обслуговує множину клієнтів, приймає, маршрутизує, зберігає та передає повідомлення між учасниками. Система повинна підтримувати обмін даними в реальному часі, коректно працювати за наявності багатьох одночасних підключень, забезпечувати цілісність повідомлень і надавати мінімальний набір засобів керування користувачами та групами.

Проєкт має бути реалізований у загальному вигляді без жорсткої прив'язки до конкретної мови програмування. Можна застосовувати будь-який фреймворк чи методологію розробки.

Архітектура проєкту повинна містити щонайменше серверний модуль і один або кілька клієнтських модулів. Сервер має забезпечувати конкурентне обслуговування клієнтів без блокування всієї системи одним повільним підключенням. Спільні структури даних сервера (список активних користувачів, групи, черги повідомлень, журнал подій, сховище офлайн-повідомлень) повинні оброблятися коректно в умовах паралельного доступу.

Студенти можуть запропонувати власну реалізацію проєкту, методів або елементів програми чи завдання, якщо вони краще демонструють проблеми із використанням багатопоточності, або якщо ці нововведення вирішують поставлену задачу симуляції краще. Всі пропозиції та девіації від вимог необхідно описувати та обґрунтовувати.

Обов'язковими вимогами є працюючий сервер і щонайменше один клієнт, демонстрація одночасної роботи кількох клієнтів, підтримка індивідуальних повідомлень та спілкування в групах, ведення переліку учасників і фіксація відключених користувачів.

Повноцінний проєкт включає в себе реалізацію розподіленої клієнт-серверної чат-системи для роботи щонайменше в межах локальної мережі (не обов'язково, але треба навести тестування у межах будь-якої мережі, для повноцінної демонстрації передачі повідомлень) або між кількома вузлами, також треба

обґрунтовано обрати й описати модель паралельної обробки з'єднань, підтримати всі обов'язкові функції – текстові повідомлення, форматування, файли, мультимедіа, приватні повідомлення, групи, перелік учасників, виявлення відключень, офлайн-повідомлення або надсилання повідомлення учасникам, що не в мережі.

Часткове виконання проекту включає в себе реалізацію серверу і клієнту для одночасної роботи кількох користувачів, підтримку текстових повідомлень, приватні повідомлення, broadcast, перелік учасників і базові групи. Також, треба реалізувати хоча б один із додаткових механізмів – передавання файлів, мультимедійні повідомлення або офлайн-доставку. Провести базовий аналіз результатів.

Мінімальний проект включає в себе реалізацію локальної чат-програми з сервером і кількома клієнтами на одному комп'ютері, підтримку реєстрації користувача, обмін текстовими повідомленнями. Треба показати базове використання сокетів або іншого мережевого інтерфейсу та конкурентного обслуговування підключень. Для демонстрації роботи достатньо розробити консольний додаток.

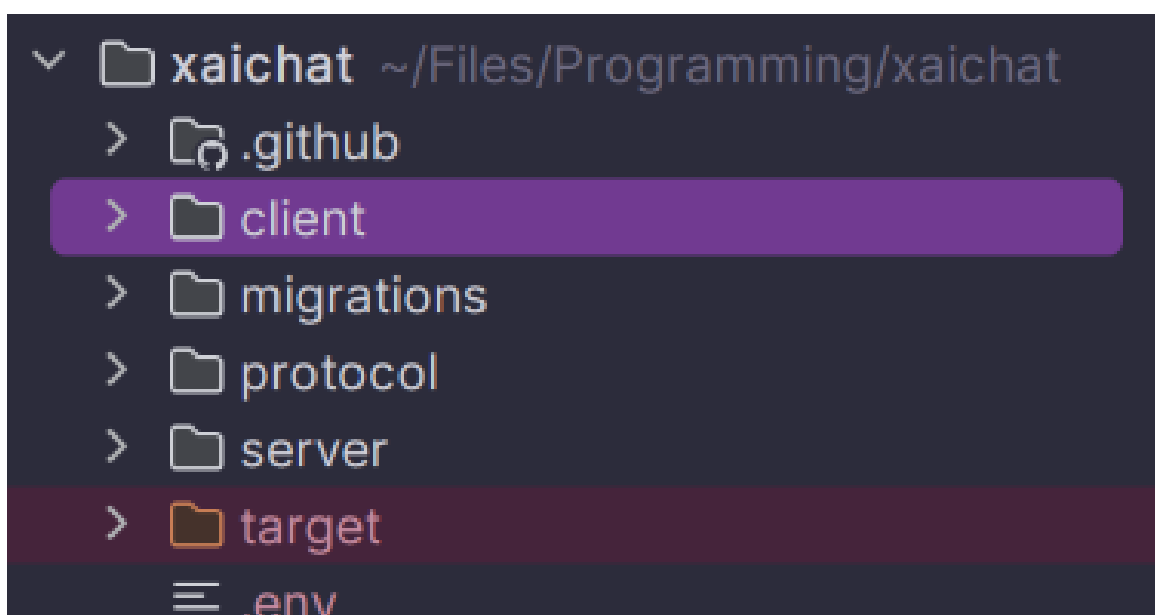
Приклад виконання проекту. Запускається сервер, який відкриває мережевий порт і очікує підключення клієнтів. Користувачі підключаються до сервера та реєструються під унікальними іменами. Кожне нове з'єднання обробляється окремим потоком, задачею або асинхронним обробником. Клієнти передають текстові, файлові або мультимедійні повідомлення одному користувачу, групі або всім учасникам. Сервер визначає одержувачів, виконує маршрутизацію, фіксує подію в журналі та контролює статус доставлення. Якщо одержувач тимчасово недоступний, сервер зберігає повідомлення для подальшої доставки. У разі втрати з'єднання сервер позначає клієнта як неактивного та оновлює перелік учасників. Після повторного підключення користувач отримує непрочитані або недоставлені повідомлення.

Хід роботи.

Для реалізації чат-системи було обрано мову програмування Rust, що зумовлено її здатністю забезпечувати безпеку роботи з пам'яттю без використання збирача сміття, а також потужними засобами асинхронного програмування. Розробка почалася з проєктування бінарного протоколу обміну даними, який став основою для взаємодії між модулями. Використання серіалізації `postcard` дозволило досягти мінімального оверхеда при передачі складних структур даних, включаючи бінарні файли.

Центральним елементом серверної частини стала асинхронна модель акто-

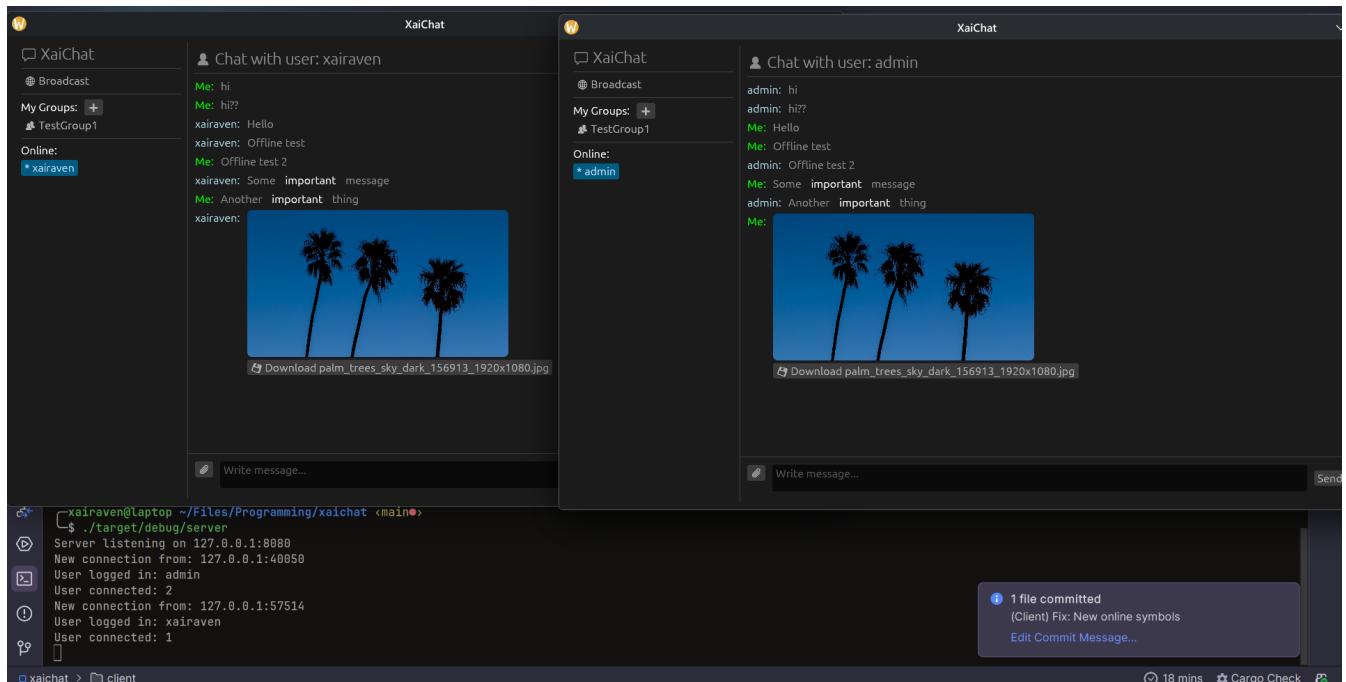
рів, реалізована за допомогою середовища `tokio`. Архітектура сервера побудована таким чином, щоб уникнути блокуючих викликів. Кожне нове TCP-з'єднання ініціює створення окремої асинхронної задачі `ConnectionActor`, яка відповідає за де-серіалізацію пакетів та їх попередню обробку. Стан усієї системи – списки активних сесій, підписки на групи та адресна книга – інкапсульований в акторі `Router`. Взаємодія між обробниками з'єднань та роутером відбувається виключно через канали передачі повідомлень (`mpsc`), що усуває потребу у використанні заблокованих структур типу `Mutex` для глобальних даних і запобігає виникненню взаємних блокувань (`Deadlocks`).



Окрему увагу було приділено цілісності даних та офлайн-доставці. Всі повідомлення, що проходять через сервер, асинхронно записуються у базу даних PostgreSQL. Для цього використано асинхронний драйвер `sqlx` з пулом з'єднань. Реалізація механізму синхронізації дозволяє клієнту після авторизації запросити всі повідомлення, створені після певного часового штампа. Сервер обробляє цей запит, витягує необхідні записи та пакує їх у повідомлення `SyncBatch`, що мінімізує кількість мережових транзакцій при завантаженні великої історії.

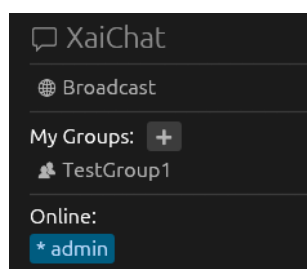
Клієнтський додаток розроблено з використанням фреймворку `egui`. Головною складністю була інтеграція асинхронної мережевої частини з синхронним циклом малювання інтерфейсу. Проблема була вирішена шляхом винесення мережевого менеджера в окремий потік ОС, який спілкується з UI-потокм через канали `crossbeam`. Для відображення мультимедійних даних, зокрема зображень, було впроваджено кеш текстур. Оскільки `egui` вимагає незмінного доступу до стану під час рендеру, кеш реалізовано з використанням механізму внутрішньої мутабельності (`RefCell`). Щоб уникнути колізій імен файлів у пам'яті, ключем у кеші

виступає комбінація часового штампа та імені файлу.

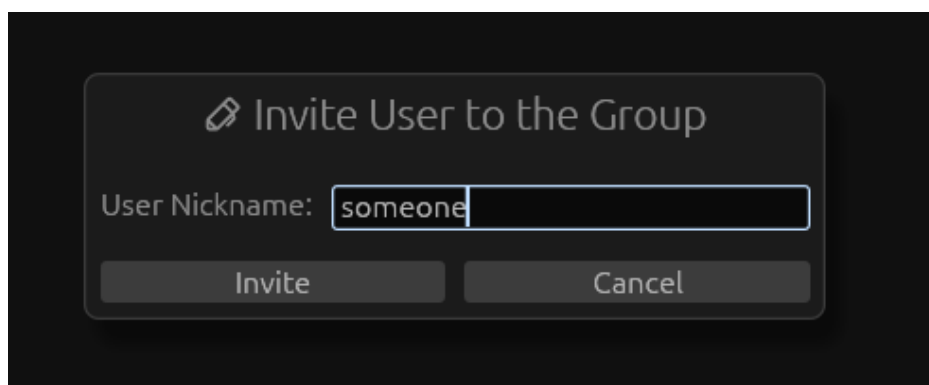
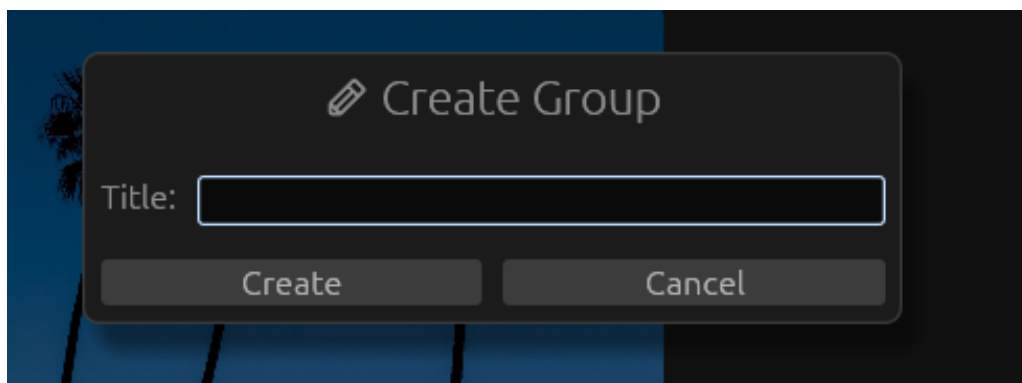


Функціонал передачі файлів реалізовано через інтеграцію з нативними діалоговими вікнами операційної системи (бібліотека `rfd`). Це дозволило користувачам обирати будь-які файли для надсилання та вказувати місце для збереження отриманих даних. Система автоматично визначає тип контенту: якщо переданий файл є зображенням, клієнт намагається декодувати його через бібліотеку `image` для негайного відображення в потоці чату, інакше – пропонує кнопку для завантаження.

Важливим аспектом стало відстеження статусів присутності користувачів (Presence). Сервер моніторить стан сокетів у реальному часі. При розриві з'єднання (навіть аварійному) обробник задачі завершується, що ініціює подію відключення в роутері. Роутер, своєю чергою, розсилає оновлений статус усім активним клієнтам. Це дозволило реалізувати в інтерфейсі адресну книгу з візуальними індикаторами онлайн-статусу. Також було вирішено проблему *TCP Reset Race Condition*: при виникненні помилки авторизації сервер тепер виконує `graceful shutdown` сокета, що гарантує доставку повідомлення про помилку клієнту до моменту фізичного розриву з'єднання.



Для підтримки форматування тексту було розроблено кастомний парсер, який обробляє розмітку для жирного шрифту. Це реалізовано через розбиття вхідного рядка на сегменти та їх послідовне відмальовування з різними стилями тексту (RichText). Маршрутизація повідомлень підтримує приватні чати, де сервер забезпечує «відлуння» повідомлення відправнику для оновлення його інтерфейсу, та групові чати на основі моделі підписок (Publish/Subscribe).



Висновок. В результаті виконання навчального проєкту було розроблено розподілену чат-систему на основі мови Rust та архітектури Actor Model. В ході роботи вдалося успішно вирішити задачі конкурентного обслуговування багатьох клієнтів без блокування всієї системи, забезпечити цілісність даних при передачі бінарних файлів та реалізувати надійне сховище для офлайн-повідомлень з використанням PostgreSQL.

Дослідження проблем мережевої взаємодії дозволило впровадити механізми м'якого завершення з'єднань та синхронізації статусів у реальному часі. Проєкт демонструє високу масштабованість завдяки асинхронній обробці вводу-виводу та ефективне використання ресурсів при рендерингу мультимедійного контенту. Всі обов'язкові функції, включаючи приватні та групові повідомлення, передачу файлів та форматування тексту, реалізовані та протестовані в умовах одночасної роботи кількох клієнтів.